Практические кейсы и примеры

Production-пайплайн, разбор инцидентов, чек-листы и шпаргалка траблшутинга

DevOps Pro — Модуль 8

Companion lab: `labs/capstone_lab.md`

Agenda

Part 1: Эталонный production-пайплайн

build → test → scan → sign (опционально) → publish → deploy → observe

Part 2: Пять разборов инцидентов

Rollout, CVE-gate, утечка секретов, медленный пайплайн, шум алертов

Part 3: Чек-листы best practices

CI, Kubernetes, безопасность, наблюдаемость, GitOps

Part 4: Шпаргалка траблшутинга

Статусы Pod-ов, kubectl, зависшие пайплайны, runner tags

Part 5: Capstone

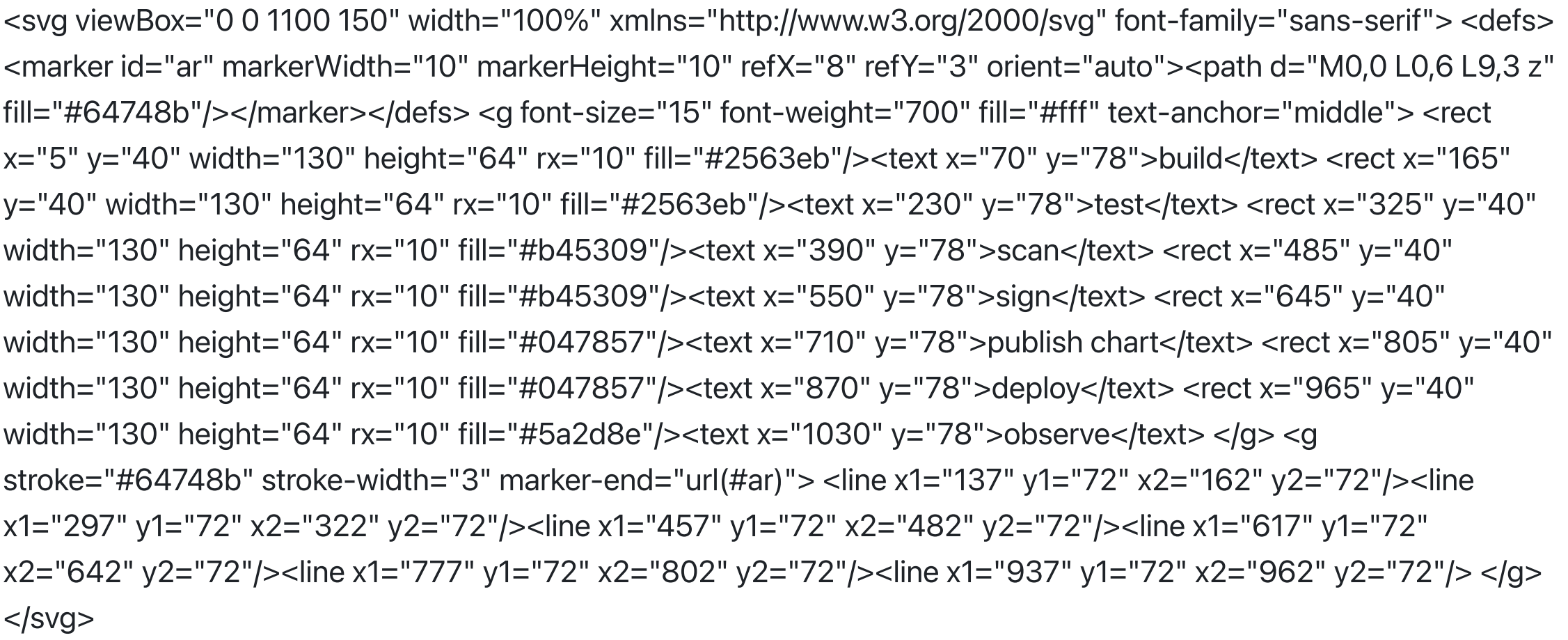
Сквозная лабораторная и критерии приёмки

Part 1: Эталонный пайплайн

От коммита до метрик в Grafana

Один пайплайн: семь стадий

> **Каждый коммит проходит одни и те же quality gates; deploy — только то, что прошло все.**



Что проверяет каждая стадия

Стадия	Что делает	Quality gate (fail = стоп)
build	собирает образ	сборка не прошла
test	unit / lint	тесты красные
scan	Trivy по образу	есть CRITICAL
sign (опционально)	Cosign keyless (OIDC)	подпись не создана, если стадия включена
publish	`helm package` в registry	lint чарта красный
deploy	`helm upgrade --install`	rollout не здоров
observe	ServiceMonitor + алерт	нет метрик после выкладки

Build и test: быстро и воспроизводимо

> Образ собираем один раз и дальше везде используем только его.

```
stages: [build, test, scan, sign, publish, deploy]

build:
  stage: build
  image: docker:27
  services: [docker:27-dind]
  tags: [docker, privileged]
  script:
    - docker build -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA .
    - docker push $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA

unit-test:
  stage: test
  image: <образ с инструментами проекта>
  needs: [build]
  script: [make test]
```

Scan и sign: цепочка поставки

> Не прошёл Trivy или нет подписи — образ не доедет до кластера.

```yaml
trivy:
  stage: scan
  image: { name: aquasec/trivy:0.65.0, entrypoint: [""] }
  script:
    - trivy image --exit-code 1 --severity CRITICAL $IMAGE

cosign:  # опционально
  stage: sign
  image: gcr.io/projectsigstore/cosign:v2.5.0
  id_tokens:
    SIGSTORE_ID_TOKEN: { aud: sigstore }
  script:
    - cosign sign --yes $IMAGE
```

Publish и deploy: через registry и агент

> **Деплоим опубликованный чарт, а не файлы из рабочей копии.**

```
variables: { KUBE_CONTEXT: "<group>/<project>:dtu-agent" }

deploy:
  stage: deploy
  image: { name: alpine/helm:4.2.0, entrypoint: [""] }
  needs: [secret-detection, trivy, publish]
  environment: { name: staging }
  script:
    - helm registry login "$CI_REGISTRY" -u "$CI_REGISTRY_USER" -p "$CI_REGISTRY_PASSWORD"
    - helm upgrade --install capstone "oci://$CI_REGISTRY_IMAGE/charts/<name>"
        --version "0.1.${CI_PIPELINE_IID}" --kube-context "$KUBE_CONTEXT"
        --namespace capstone --create-namespace
        --set image.tag="$CI_COMMIT_SHORT_SHA"
        --rollback-on-failure --timeout 3m --wait
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
```

- Доступ в кластер: **GitLab Kubernetes Agent**. `KUBE_CONFIG` не используем: kubeconfig в CI-переменных даёт админский доступ любому, кто запускает pipeline.
- `--rollback-on-failure` откатывает релиз, если он не стал здоровым; `--version` берёт версию, которую собрал этот pipeline

Observe: деплой не закончен, пока не видны метрики

> «Выкатили» = ServiceMonitor подхвачен и алерт на ошибки живой.

```yaml
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: app
  labels: { release: monitoring }
spec:
  selector: { matchLabels: { app: app } }
  endpoints: [{ port: http, path: /metrics }]
```

- Label `release: monitoring` обязателен и на ServiceMonitor, и на PrometheusRule: без него Prometheus их не подхватит
- Проверка в Prometheus: target в статусе **UP**
- Алерт на ошибки привязан к SLO (см. Part 2, кейс 5)

Part 2: Пять разборов инцидентов

Симптом → диагностика → причина → исправление → профилактика

Как читать разбор

> **Сначала факты (симптом и диагностика), потом гипотеза, и только потом фикс.**



Кейс 1: релиз завис на половине

> **Новая версия не становится Ready, а старая ещё держит трафик.**

Симптом: `helm upgrade` висит, часть Pod-ов в `0/1 Running`.

```
kubectl rollout status deploy/app
kubectl get pods -l app=app
kubectl describe pod <new-pod>     # Events: Readiness probe failed
kubectl logs <new-pod>             # readiness-проба не перезапускает контейнер: --previous пуст
```

Кейс 1: причина, фикс, профилактика

> Readiness probe смотрела на неверный путь или порт.

Причина	probe бьёт в `/health`, а приложение отдаёт `/healthz`
Фикс	поправить `path` / `port` в `values.yaml`, `helm upgrade`
Откат	`helm rollback <release> <revision>`: релиз ведёт Helm, `kubectl rollout undo` обходит его и расходится с `helm history`
Профилактика	`--rollback-on-failure` в CI + проверка probe в staging

Кейс 2: CVE quality gate остановил релиз

> **Trivy нашёл CRITICAL, пайплайн упал на scan — и это правильное поведение.**

Симптом: вчера зелёный пайплайн сегодня красный, код не менялся.

```
trivy image --severity CRITICAL $IMAGE      # какие пакеты
trivy image --format json -o report.json $IMAGE
```

Новая уязвимость в базе данных Trivy или в базовом образе — код ни при чём.

Кейс 2: как выходить, не выключая quality gate

> **Обновляем базовый образ; игнор — только осознанный и с датой.**

1. Обновить базовый образ или зависимость до версии с исправлением
2. Нет исправления — `.trivyignore` с комментарием, тикетом и датой пересмотра
3. Не менять `--exit-code 1` на `0` ради «зелёного» цвета
4. **Профилактика:** регулярная пересборка и плановое сканирование, а не только на push

Кейс 3: секрет попал в Git

> **Токен в коммите считается скомпрометированным, даже если коммит удалили.**

Симптом: секрет-сканер нашёл ключ; или пайплайн печатает переменную в лог.

```
git log -S 'AKIA' --all          # найти, где появился
gitleaks detect --source .       # проверить репозиторий
```

- В Git история неизменна: ключ уже мог скопироваться
- В логах job-а маскируются только переменные с флагом **Masked**

Кейс 3: отзыв, ротация, профилактика

> **Порядок действий: revoke → rotate → clean → prevent.**

Шаг	Действие
revoke	отозвать скомпрометированный токен/ключ
rotate	выпустить новый, положить в **protected + masked** variable
clean	убрать из истории, закрыть доступ к форкам и кэшам
prevent	secret detection в MR-пайплайне, short-lived credentials, scoped tokens

Кейс 4: пайплайн идёт 25 минут

> **Узкое место почти всегда одно из трёх: нет cache, нет `needs`, лишние запуски.**

Симптом: разработчики ждут, MR не мержатся.

```
# Во вкладке Pipelines → Job durations
# Что занимает время: install deps? docker build? ожидание runner-а?
```

Подозрение	Признак
нет cache	каждый job скачивает зависимости заново
стадии идут строго по очереди	job ждёт всю предыдущую стадию
запуск на каждое изменение	правка README гоняет все тесты

Кейс 4: три рычага ускорения

> **cache + needs + rules:changes обычно дают самый большой выигрыш.**

```yaml
test:
  needs: [build]                # DAG: стартует сразу после build
  cache:
    key: { files: [package-lock.json] }
    paths: [node_modules/]
  rules:
    - changes: ["src/**/*", "package-lock.json"]
```

- **Профилактика:** монорепо — `include:` + `rules: changes:` на каждый сервис
- Параллельные job-ы ограничены числом runner-ов

Кейс 5: алерты шумят — на них уже не смотрят

> **Алерт «CPU > 80%» срабатывает, когда пользователям всё хорошо.**

Симптом: десятки уведомлений за ночь, дежурный их отключил.

- Пороговые алерты ловят **причины**, а не влияние на пользователя
- Реальный инцидент теряется в шуме
- Нужно оповещать о скорости **сгорания error budget**

Кейс 5: SLO и burn rate

> **Будим человека, только если бюджет ошибок сгорает быстро.**

SLO 99.9% → бюджет ошибок `0.001`. Burn rate = доля ошибок / `0.001`.

```
- alert: APIServerErrorBudgetFastBurn
  expr: |
    apiserver:request_errors:ratio_rate5m / 0.001 > 14.4
    and apiserver:request_errors:ratio_rate1h / 0.001 > 14.4
```

- 14.4× за короткое **и** длинное окно → страница дежурному
- Медленный burn → тикет, а не ночной звонок

Part 3: Чек-листы best practices

Пять списков — по одному на область

Чек-лист: CI

> **Пайплайн — это код: он ревьюится, версионируется и быстро даёт ответ.**

- [ ] `rules:` вместо устаревших `only/except`
- [ ] `cache:` для зависимостей, `needs:` для DAG
- [ ] Образ тегируется по SHA, не `latest`
- [ ] Общие куски вынесены в `include:` / CI/CD Components
- [ ] У каждого job-а явный `image:`
- [ ] Монорепо: `rules: changes:` на сервис

Чек-лист: Kubernetes

> **Если у Pod-а нет probe и лимитов, Kubernetes не может его защитить.**

- [ ] `readinessProbe` и `livenessProbe` настроены осознанно
- [ ] `requests` и `limits` заданы, нет «голых» Pod-ов
- [ ] Деплой через Deployment, не одиночные Pod-ы
- [ ] `PodDisruptionBudget` для критичных сервисов
- [ ] Конфиги в ConfigMap, секреты в Secret, не в образе
- [ ] RBAC по минимуму, NetworkPolicy между namespace-ами

Чек-лист: безопасность

> **Безопасность встроена в пайплайн, а не добавляется перед релизом.**

- [ ] Secret detection (gitleaks) и SAST (Semgrep) в MR
- [ ] Trivy по образу с падением на CRITICAL
- [ ] Образы подписаны (Cosign keyless, опционально); проверка подписи при admission вне курса
- [ ] Секреты только в protected + masked variables
- [ ] Short-lived credentials и OIDC вместо долгоживущих ключей
- [ ] SecurityContext и Pod Security Admission в кластере

Чек-лист: наблюдаемость

> **Алерт имеет смысл, только если на него есть действие.**

- [ ] У каждого сервиса есть `ServiceMonitor` и метрики
- [ ] Алерты по SLO и burn rate, а не по сырым порогам
- [ ] Alertmanager: маршрутизация и silences настроены
- [ ] Дашборды как код (Grafana provisioning)
- [ ] Для расследования есть логи и трейсы (Loki + Alloy, Jaeger)

Чек-лист: GitOps

> **Git — единственный источник истины, кластер только его отражает.**

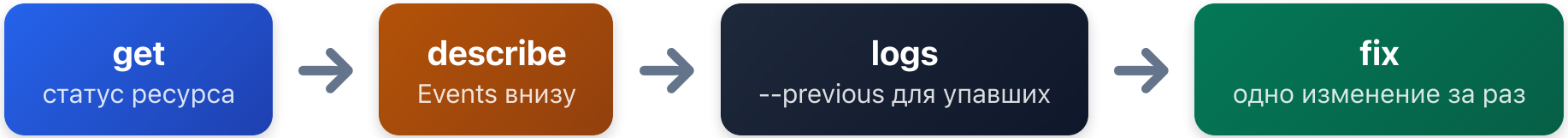
- [ ] Изменение = commit / PR, ручных правок в кластере нет
- [ ] Агент (Argo CD) сам синхронизирует и чинит дрейф
- [ ] Откат = `git revert`
- [ ] Окружения описаны в репозитории, а не в голове
- [ ] CI не держит админский доступ в кластер

Part 4: Шпаргалка траблшутинга

Что смотреть, когда что-то не работает

Порядок диагностики

> **Идём от общего к частному: статус → события → логи.**



kubectl: пять команд на все случаи

> **Эти команды решают большинство разборов.**

Команда	Зачем
`kubectl get pods -o wide`	статус, нода, перезапуски
`kubectl describe pod <p>`	Events: probe, образ, scheduling
`kubectl logs <p> --previous`	лог упавшего контейнера
`kubectl get events --sort-by=.lastTimestamp`	что происходило в namespace
`helm rollback <release> <rev>`	откат Helm-релиза (не `kubectl rollout undo`)

Статусы Pod-а: что означают

> Статус называет симптом; причину ищем в `describe`.

Статус	Типичная причина	Куда смотреть
`Pending`	нет ресурсов, taints, PVC не привязан	Events: scheduling
`ImagePullBackOff`	неверный тег, нет доступа к registry	Events, imagePullSecrets
`CrashLoopBackOff`	приложение падает при старте	`logs --previous`
`OOMKilled`	память выше `limits`	`describe`, поднять limit
`0/1 Running`	не проходит readiness probe	probe path/port

Пайплайн: зависает или падает

> Если job в `pending` — смотрим runner; если упал — читаем лог job-а.

Симптом	Причина	Фикс
job вечно `pending`	нет runner-а с нужными tags	выровнять `tags:` и runner
`no image specified`	у runner-а нет default image	задать `image:` в job-е
`docker: not found`	в образе job нет docker CLI	задать образ с docker CLI в `image:` и `services: [docker:<тег>-dind]`
`Cannot connect to the Docker daemon`	runner не privileged, DinD не запустился	privileged runner (`tags: [docker, privileged]`)
deploy: `Unauthorized`	токен/kubeconfig истёк	проверить агент или `KUBE_CONFIG`

Runner tags: как job находит исполнителя

> **Job запускается только на runner-е, у которого есть все его tags.**

```
build-image:
  tags: [docker, privileged]     # нужны оба
  script: [docker build .]
```

- Нет runner-а со всеми tags → job остаётся `pending`
- Executor определяет среду: `shell`, `docker`, `kubernetes`
- Проверить: **Settings → CI/CD → Runners** (онлайн? есть tags?)

Part 5: Capstone

Всё вместе на одном приложении

Capstone: что строим

> **Одно приложение проходит весь путь: код → образ → кластер → метрики.**



Пошаговая инструкция: `./labs/capstone_lab.md`

Capstone: критерии приёмки

> **Готово, когда вы можете показать доказательство для каждого пункта.**

Критерий	Доказательство	Баллы
Pipeline: lint, build, secret-detection, Trivy с порогом	все job зелёные; красный прогон secret-detection и trivy	15
Чарт публикуется из CI, версии закреплены	`helm show chart` для опубликованной версии	10
Deploy через Agent, environment, автооткат	Operate → Environments, `helm history`	15
Probes, requests и HPA в чарте	`kubectl get hpa` без `<unknown>`	10
ServiceMonitor, алерт и receiver	target UP, алерт Firing, сообщение в receiver	10
Логи в Loki	Grafana Explore по `{namespace="capstone"}`	10
F1, F2, F3	разбор по четырём пунктам на каждую поломку	30

Полные условия, штрафы и шкала: capstone_lab, раздел «Оценка».

Итоги и дальнейшие шаги

Что взять с собой

- **Пайплайн с quality gates** на каждой стадии, deploy только проверенного
- **Шаблон разбора:** симптом → диагностика → причина → фикс → профилактика
- **Чек-листы** как основа для код-ревью и аудита
- Дальше: GitOps с Argo CD, Helmfile, SLO для своих сервисов